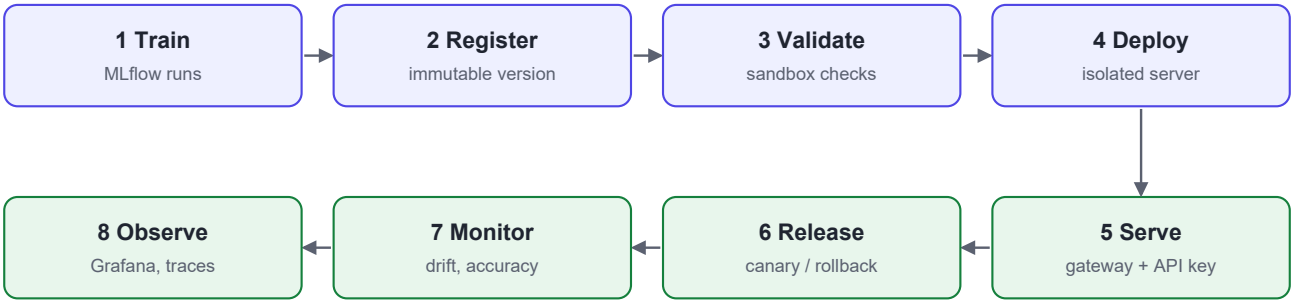
ModelOps Cloud

The end-to-end pipeline, explained, and how to run all of it



Stages 1–4 turn a trained model into a running, validated service; 5–8 keep it safe and visible in production.

What this guide covers	
Part 1 · Understand	What the platform does, its components, and every pipeline stage
Part 2 · Run	Setup, start-up, and a complete walkthrough in the web console and Grafana
Part 3 · Go further	Kubernetes, tests and benchmarks, troubleshooting, shutting down

Everything runs on one Windows laptop with free tools; nothing here costs money or needs a cloud account. Commands are written for **PowerShell**, run inside the `modelops-cloud` folder. Guide written 12 November 2026.

Part 1 · Understand

1.1 What ModelOps Cloud does

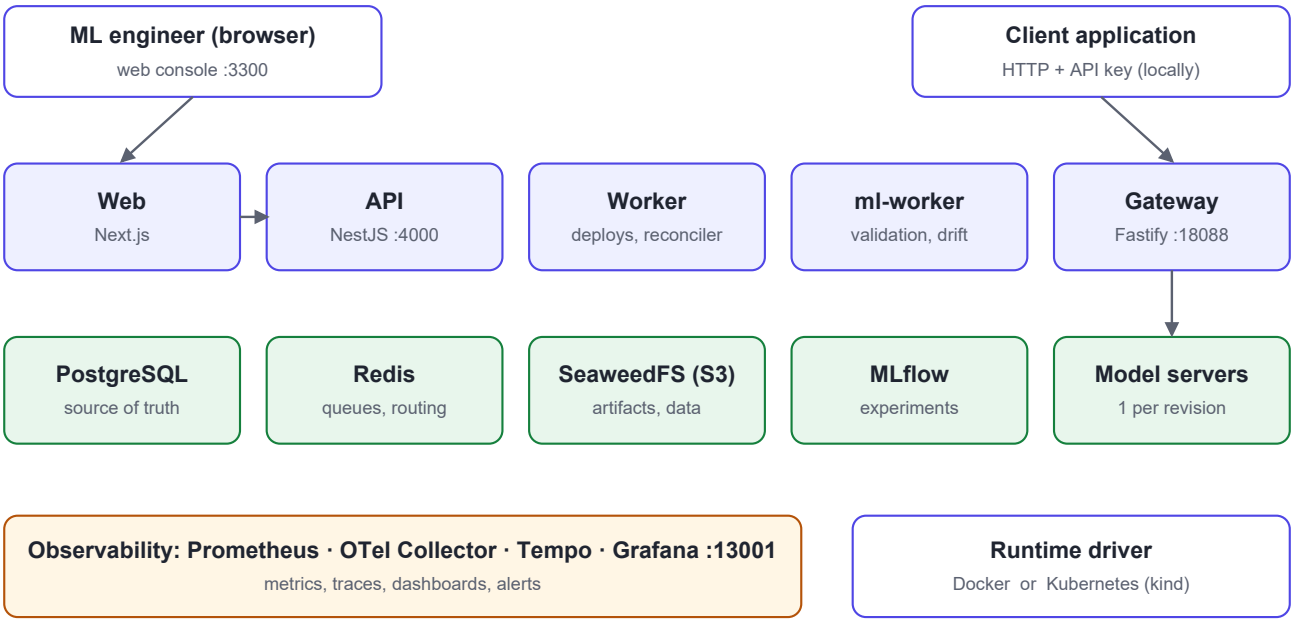
ModelOps Cloud is a self-service platform that takes a trained machine-learning model and runs it safely as a live prediction service. A data scientist can train a good model; turning it into something an application can call reliably usually means hand-writing an API, packaging dependencies, provisioning servers, wiring up monitoring, and hoping a bad release can be undone. ModelOps Cloud makes that a governed, repeatable path: every model is checked before it may serve, every deployment is durable and reversible, every request is authenticated and validated, and the whole system is observable.

Two kinds of people use it, through two different tools:

Who	Tool	What they see
ML engineers (the users)	**ModelOps web console** (port 3300)	Only their organization: projects, models, versions, endpoints, releases, their endpoints' traffic, audit log
Platform operators	**Grafana** (port 13001)	Everything, across all organizations: platform health, traces, pod resources, alerts

They are separate on purpose. Grafana reads all tenants' data, so customers must never use it; the console shows each user only their own organization, through queries the API builds for them.

1.2 The components



How they talk. Browser → Web → API. The API stores state in PostgreSQL, puts jobs on Redis queues and files in SeaweedFS, and reads MLflow runs. The worker takes deployment jobs from Redis and starts or stops model servers through the runtime driver; ml-worker validates uploaded models and computes drift. Client applications call only the gateway, which checks the API key and forwards each request to a model server. Everything reports metrics and traces to the observability stack.

Component	Role
Web console (Next.js)	The user interface. Calls the API through its own origin, so cookies stay first-party.
API (NestJS)	Accounts, organizations and roles, projects, the model registry, endpoints, releases, API keys, audit log.
Worker	Background jobs: runs deployments, the reconciler (self-healing), canary analysis, drift checks.

Component	Role
ml-worker (Python)	The sandboxed validator for uploaded models; drift detection (Evidently) and evaluations.
Gateway (Fastify)	The public inference API. Checks API keys, rate-limits, validates inputs, routes to model servers.
Model servers (FastAPI + ONNX Runtime)	One isolated, resource-limited server per deployed revision.
PostgreSQL	The source of truth for everything, including a hash-chained audit log.
Redis + BullMQ	Job queues and the gateway's routing snapshot and API-key directory.
SeaweedFS (S3-compatible)	Model artifacts (stored by SHA-256), datasets, captured inputs, reports.
MLflow	Experiment tracking: training runs, metrics, the model registry mirror.
Prometheus, OTel Collector, Tempo, Grafana	Metrics, traces, dashboards and alert rules for operators.

1.3 The pipeline, stage by stage

Stage 1 · Train

Models are trained in Python (scikit-learn and PyTorch) on a seeded, synthetic customer-churn dataset, so results are reproducible. Each training run is recorded in MLflow under the project's experiment, with its metrics, parameters and the SHA-256 of its data. Training exports an **ONNX bundle**: the model file, a signature (input names and types) and golden examples (inputs with the outputs the model must reproduce).

Stage 2 · Register

A run (or an uploaded bundle) becomes a numbered, **immutable model version**. The artifact is stored content-addressed (`sha256/<hash>`), so it can never be silently replaced, and its lineage (which data, which run) is kept.

Stage 3 · Validate

Before a version may ever serve, the ml-worker checks it in a sandbox: it must be ONNX, not a pickle that could execute code; it may use only allowed operators and no external data files; it must match its signature, reproduce its golden examples within tolerance, load within a time and memory budget, and still match its checksum. The result is a live checklist in the UI. Unvalidated versions cannot be deployed; the API and a database trigger both refuse.

Stage 4 · Deploy

Deploying creates a **revision**. The request is durable: written to Postgres with an outbox entry in the same transaction, then picked up by the worker, so it cannot be lost. The revision moves through a state machine (Requested, Queued, Provisioning, Loading, Verifying, Ready). The worker starts a hardened model server (non-root, read-only filesystem, no Linux capabilities, CPU, memory and process limits, internal network only), which downloads its artifact through a short-lived link, verifies the checksum, and must answer the golden examples correctly before it receives any traffic. A reconciler keeps reality in line with the database: it restarts crashed servers within a budget and retires old ones.

Stage 5 · Serve

Clients call `POST /v1/endpoints/<slug>/predict` on the gateway with an API key. The gateway checks the key (stored only as a hash, revocable in seconds), applies the endpoint's rate limit, validates the inputs against the model's signature, and routes to the serving revision. Responses carry the model version, the revision and a trace id.

Stage 6 · Release

A new version can start as a **canary** on 0–50 % of traffic. Every 15 seconds the worker compares the canary's error rate and p95 latency with the stable version's, and aborts a canary that is worse, on its own. **Promote** (allowed only after a passing check) makes the canary the stable version; the previous one stays warm for 30 minutes, so **Roll back** switches traffic back in milliseconds. Production endpoints additionally require a second person's approval (four-eyes).

Stage 7 · Monitor

Each endpoint has a traffic dashboard (requests by model version, latency percentiles, errors). With input capture turned on, drift checks compare live inputs with the training data; with ground-truth labels fed back, the platform measures real accuracy, F1 and ROC-AUC. A version can only become the **champion** by passing an evaluation gate against the current champion on held-out data.

Stage 8 · Observe and operate

Operators see the whole system in Grafana: platform and endpoint dashboards, traces that follow one request from the gateway into the model server, pod CPU and memory on Kubernetes, and six alert rules, each with a runbook. Every action by every user is in the hash-chained audit log, which the app can verify for tampering.

1.4 What makes it reliable and safe

- **Durable, idempotent deployments:** a transactional outbox, idempotency keys and compare-and-set state changes mean a crash or a repeated click never loses or duplicates a deployment.
- **Tested under real faults:** a chaos suite kills the worker, flushes and restarts Redis, pauses Postgres and kills model servers; every deployment still ends in the right state.
- **Security tested, not assumed:** 16 security requirements (pickles refused, tenant isolation, CSRF, API-key handling, no inputs in logs, and more) each have an automated test.
- **Two runtimes, one engine:** the same deployment logic and the same tests run on Docker and on Kubernetes (kind), where each model is a pod with a NetworkPolicy.
- **Supply chain:** CI pins every action by commit and every image by digest, scans images with Trivy, and publishes them with an SBOM; Dependabot proposes updates after a 7-day wait.
- **Zero cost:** the AWS deployment exists only as Terraform that is linted and policy-checked in CI and never applied.

Part 2 · Run

2.1 One-time setup

Installed already on this laptop: Docker Desktop, Node 24 (through **fnm**), pnpm 12, uv (Python), Git. One thing to do once: let PowerShell find Node and pnpm.

1. Open PowerShell and run `notepad $PROFILE` (create the file if Notepad asks).
2. Add the line below, save, close Notepad, and open a **new** PowerShell window.

```
fnm env --use-on-cd --shell powershell | Out-String | Invoke-Expression
```

Then, inside the `modelops-cloud` folder, install dependencies and create the secrets file (both are safe to repeat):

```
cd "C:\Users\karta\Downloads\Claude Work\modelops-cloud"
pnpm install
pnpm env:init
```

If `pnpm` is 'not recognized', the profile line above is missing or the window is old: open a new PowerShell window.

2.2 Start everything

Use two PowerShell windows, both in the `modelops-cloud` folder.

Window 1: infrastructure (finishes and returns)

```
pnpm dev:up        # Postgres, Redis, S3, MLflow, validator, gateway, Prometheus (waits until healthy)
pnpm db:deploy     # database migrations
pnpm dev:obs       # tracing and Grafana
```

Window 2: the application (keeps running)

```
pnpm dev           # API, worker and web console; leave this window open
```

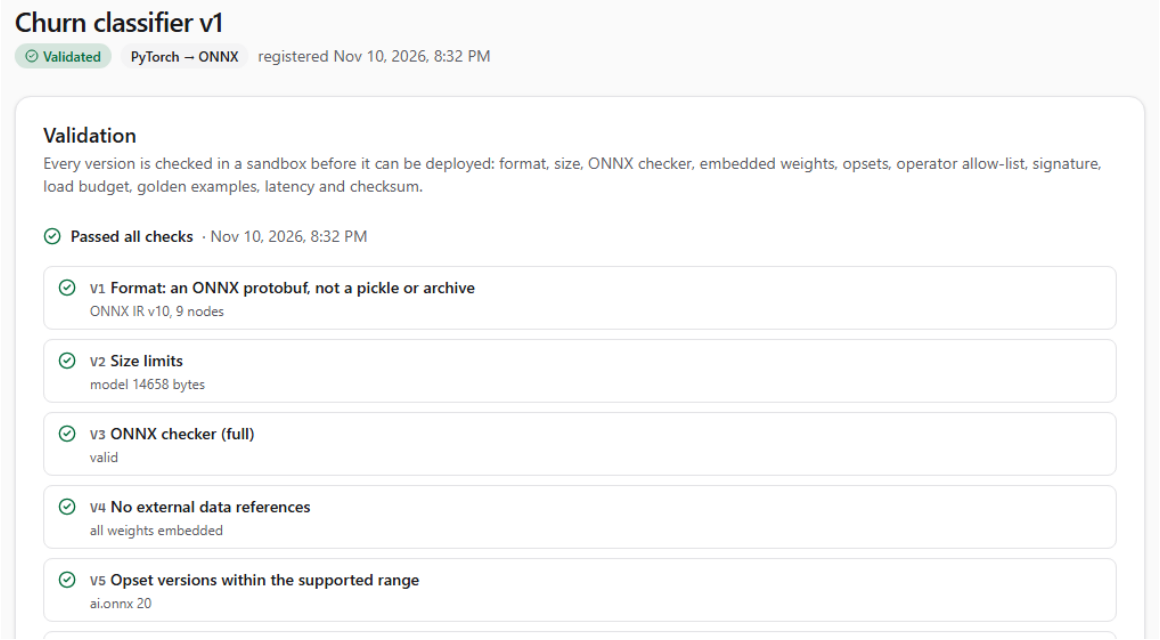
When `http://127.0.0.1:3300` opens in a browser, everything is up. The addresses you will use:

What	Address	Notes
Web console	http://127.0.0.1:3300	Create an account the first time
API and its documentation	http://127.0.0.1:4000/api/docs	Swagger UI
Inference gateway	http://127.0.0.1:18088	Clients call this with an API key
Grafana	http://127.0.0.1:13001	User admin; password in .env (see 2.6)
Prometheus (alerts)	http://127.0.0.1:19090/alerts	The six alert rules
MLflow	http://127.0.0.1:15000	Experiments and runs

2.3 From a model to a live endpoint (web console)

1. Open **http://127.0.0.1:3300**, choose **Create account**, register. You get your own workspace (organization) as its Owner.
2. **Projects** → **New project**, name it, e.g. *Churn*, and open it.
3. **New model**, name it *Churn classifier*, and open it.
4. **Upload bundle**: choose the three files in `tests\fixtures\bundles\sklearn\` (model.onnx, signature.json, golden_examples.json), then **Upload and validate**. Wait for **Passed all checks**.
5. Back in the project: **New endpoint**, name it *Churn API*. On its page, **Deploy** → choose version 1 → **Deploy**. Watch the steps until **Ready**.

6. Scroll to **Playground** and press **Predict**: the first live prediction, with the version, latency and revision that answered.



A validated version: it passed all 11 sandbox checks (the first five shown).

Training your own models instead of the sample bundle (optional). The project page shows the exact command for its MLflow experiment. It trains a scikit-learn and a PyTorch model and records both runs; open a run in the project and choose **Register version**.

```
uv sync --directory ml
pnpm ml:train --experiment modelops/<org-slug>/<project-slug>
```

2.4 Call it like a real application

1. In the project: **New API key**. Copy the key (it starts with `mo_`); it is shown only once.
2. In PowerShell (replace the key; the endpoint's slug is shown under its name, e.g. `churn-api`):

```
$key  = "mo_PASTE_YOUR_KEY"
$body = '{"inputs":[{"tenure_months":14,"monthly_charges":72.5,"support_tickets":3,
          "contract_type":1,"late_payments":0,"usage_hours":41.2}]}'
$r = Invoke-WebRequest -Method Post -UseBasicParsing `
     -Uri "http://127.0.0.1:18088/v1/endpoints/churn-api/predict" `
     -Headers @{ Authorization = "Bearer $key" } -ContentType "application/json" -Body $body
$r.StatusCode; $r.Headers["x-model-version"]; $r.Headers["x-trace-id"]; $r.Content
```

Expect **200**, the model version, a **trace id** (keep it for 2.7) and the prediction: `label` 0 means the customer is predicted to stay, 1 to churn, with a probability for each.

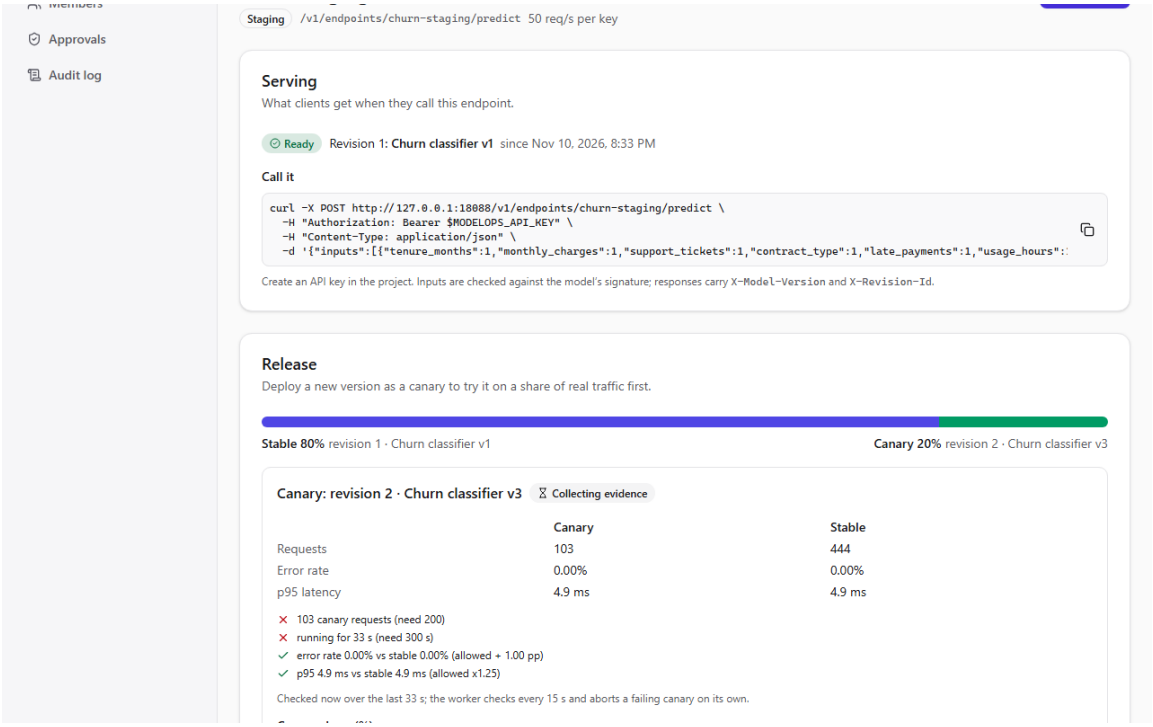
In PowerShell, `curl` is not curl but an alias of Invoke-WebRequest, which is why the command above uses Invoke-WebRequest. (In Git Bash, real curl works.) Other answers: 401 wrong or revoked key · 404 wrong slug or another project's key · 422 inputs don't match the signature · 429 rate limit.

To send steady traffic (needed for the canary check and for the charts), run for 6 minutes at about 20 requests per second:

```
pnpm demo:traffic --endpoint churn-api --key mo_PASTE_YOUR_KEY --rps 20 --duration 6m
```

2.5 A safe release: canary, promote, roll back

1. On the model: **Upload bundle** again with `tests\fixtures\bundles\pytorch\`; it becomes version 2 and is validated.
2. On the endpoint: **Deploy** → version 2 → rollout **Canary: a share of traffic first** → share **20** → **Deploy**. The **Release** panel shows *Stable 80 % / Canary 20 %*.
3. Start the traffic command from 2.4. The canary panel fills in: requests, error rate and p95 latency for canary and stable side by side.
4. The check needs **at least 200 canary requests and 5 minutes**. Until then it reports *inconclusive*. Playground clicks do not count; only real API traffic does.
5. When the check passes, **Promote** becomes available: version 2 then serves 100 %. A failing canary is aborted by the worker automatically.
6. Press **Roll back** and confirm: traffic returns to version 1 within milliseconds, because it is still warm.
7. Open **Audit log** → **Verify integrity**: every action above, who did it and when, and the result **Intact**.

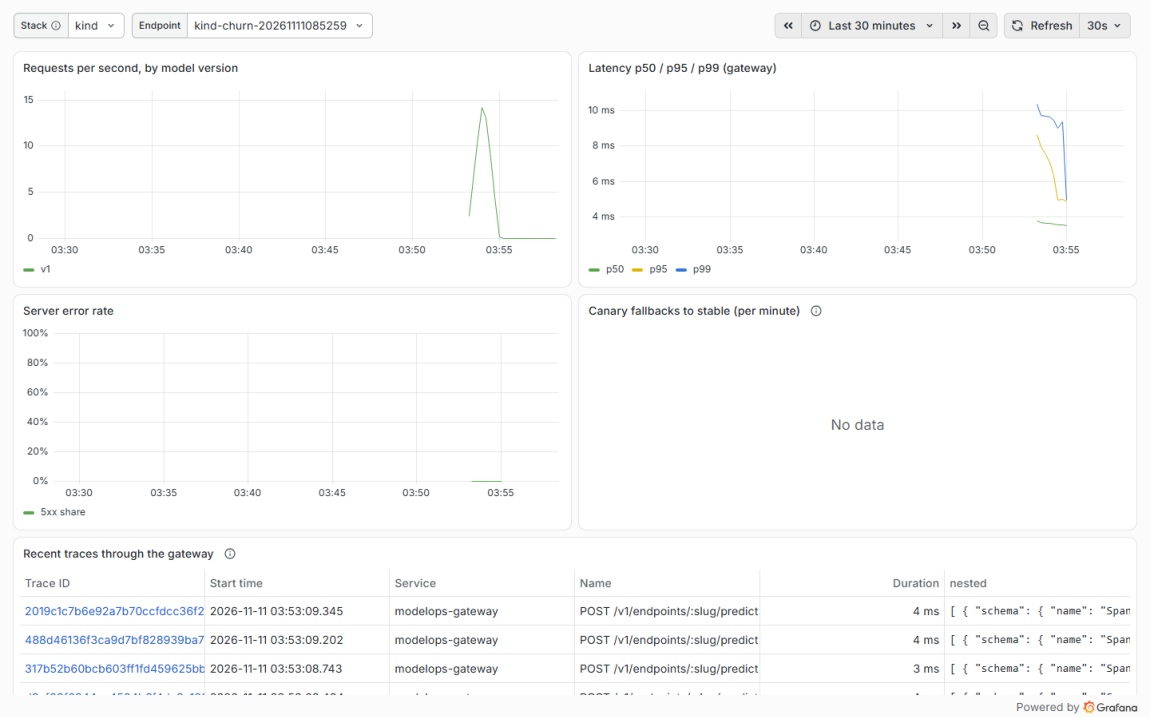


The Release panel during a canary: the split, the live comparison and the checks.

2.6 Grafana: the operator's view

1. Find the Grafana password: `Select-String MO_GRAFANA_ADMIN_PASSWORD .env` (copy the part after =).
2. Open **http://127.0.0.1:13001** and log in as **admin**.
3. **Dashboards** → **ModelOps · Endpoint**. At the top choose **Stack: dev** and your **Endpoint** (e.g. churn-api), time range **Last 15 minutes**. You see requests by model version (the canary hand-over is visible), latency p50/p95/p99, server error rate, canary fallbacks and recent traces.
4. **Dashboards** → **ModelOps · Platform**: endpoints serving, deployments in progress, approvals waiting, recent deployments, failures by code.
5. **ModelOps · Drift** shows drift reports; **ModelOps · Kubernetes pods** fills in when the kind cluster runs (Part 3).

> The Endpoint list is alphabetical and includes old benchmark endpoints; pick yours, or the charts stay flat.



Grafana's Endpoint dashboard: traffic by model version, latency, errors and the traces of recent requests.

2.7 Follow one prediction from end to end (tracing)

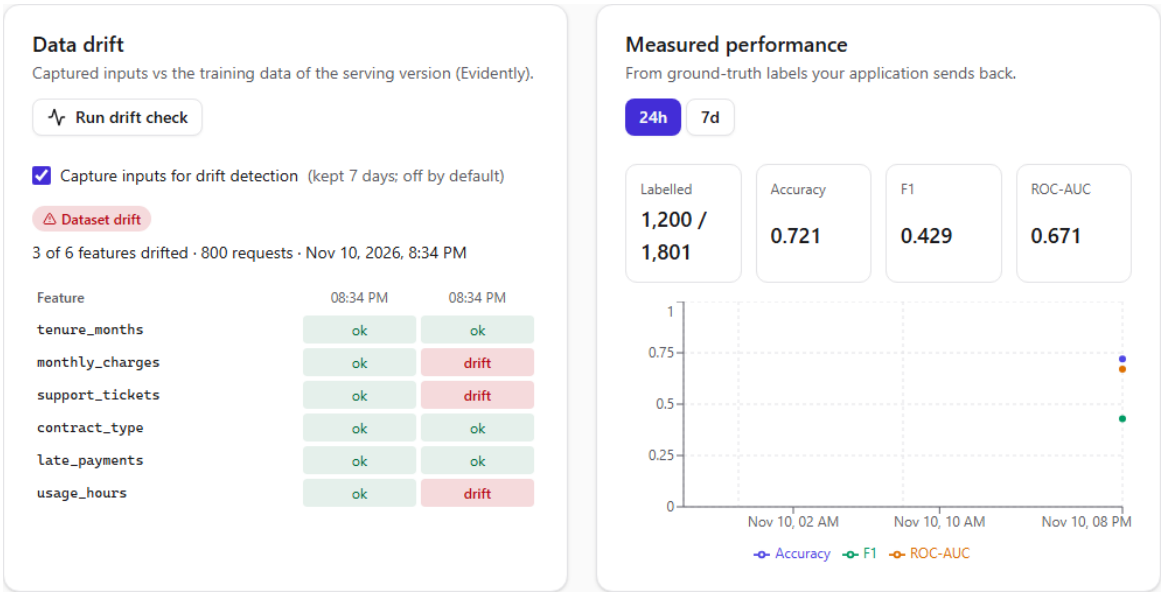
1. In Grafana: **Explore** (compass icon) → data source **Tempo** → query type **TraceQL**.
2. Paste the `x-trace-id` from 2.4 and run the query.
3. One trace: the gateway's span for the request, its call to the model server, and the model server's own span, each with its duration. This is how an operator finds where a slow request spent its time.

2.8 Monitoring: drift and the evaluation gate (optional)

1. On the endpoint, turn on **Capture inputs** (off by default for privacy; kept 7 days).
2. Send predictions with true labels, normal data first, then drifted data:

```
pnpm demo:drift --endpoint churn-api --key mo_PASTE_YOUR_KEY --phase normal
pnpm demo:drift --endpoint churn-api --key mo_PASTE_YOUR_KEY --phase data
```

1. Press **Run drift check**: a heatmap shows which input features drifted from the training data, next to the measured accuracy.
2. **Evaluation gate:** upload a held-out dataset in the project's **Datasets** section; on the model, **Evaluate** compares a candidate version with the current champion on it. Only a passing evaluation can make a version **champion**.



Drift and measured performance side by side on an endpoint.

2.9 Alerts

Open **http://127.0.0.1:19090/alerts**. Six rules watch the platform: gateway down, high server-error rate, high latency, canary falling back, prediction events dropped, rate limiter failing open. Each has a runbook in `docs\runbooks\` saying what it means, how to confirm it and what to do.

Part 3 · Go further

3.1 The same platform on Kubernetes (kind)

The platform can also run on a local Kubernetes cluster, where each deployed model becomes a hardened pod with its own NetworkPolicy (only the gateway may reach it; it may reach object storage but not the database). The data services stay in Docker; the cluster joins their network.

```
pnpm kind:up            # ~5 min the first time: cluster, images, database, deploy, wait
pnpm kind:smoke         # 12 end-to-end checks against the cluster
pnpm kind:down          # delete the cluster
```

On kind	Address
Web console	http://127.0.0.1:13400
API	http://127.0.0.1:14400
Gateway	http://127.0.0.1:18188
Grafana	the same Grafana; choose Stack: kind and Database: ModelOps DB (kind)

The kind platform is a separate instance with its own database and accounts. Inspect it with `kubectl --context kind-modelops get pods -A`.

3.2 Tests and checks

Command	What it proves
pnpm test	Unit and component tests for the API, gateway and web console
pnpm test:integration	The real services together: deployments, gateway, drift, security tests, tracing
pnpm --filter @modelops/api test:k8s	The same lifecycle tests on Kubernetes, with network-policy probes (needs kind:up)
pnpm test:chaos	Real faults during deployments: killed worker, Redis flush/restart, paused Postgres
pnpm test:e2e	Browser journeys with Playwright, including real model training and accessibility checks
pnpm test:smoke	25 quick environment checks; the first thing to run when something seems broken
pnpm check:hardening	What Docker actually applied to every running container
pnpm tf:check	The AWS Terraform: fmt, validate, tflint, checkov. Never applied; needs no AWS account
pnpm bench, then pnpm bench:report	Load tests (about 15 min) and the benchmark report generated from their raw results

The same checks run in GitHub Actions on every push; the latest run on `main` is green.

3.3 What was measured

Three full benchmark runs on this laptop, one model server at its default size (0.5 CPU, 512 MiB). Every number comes from a raw result file in `benchmarks\results\`; the full report is `docs\benchmarks\report.md`.

Measurement	Runs 1 / 2 / 3
Failed requests at 50 requests/second for 3 minutes	0 / 0 / 0
Time the gateway adds at the median (vs calling the model server directly)	1.45 / 3.27 / 3.48 ms

Measurement	Runs 1 / 2 / 3
Highest rate one default-sized model server sustained	100 / 50 / 200 requests/second
Share of traffic a 10 % canary received	9.88 / 10.58 / 10.56 %
Rollback, request to the gateway serving the old version (median of 10)	54.0 / 113.8 / 61.6 ms
Memory of all containers with two model servers	2.15 / 2.45 / 2.80 GiB

Latency tails (p95, p99) varied a lot between runs because the laptop was busy with other work; the report explains it with data. A fix found by these benchmarks (the reconciler no longer restarts a model server that is busy rather than dead) answered 12.3 % more requests under overload.

3.4 Troubleshooting

Symptom	Cause and fix
'pnpm' / 'node' is not recognized	fnm is not set up in this PowerShell window: see 2.1, open a new window.
'grep' is not recognized	PowerShell has no grep: use Select-String, e.g. Select-String MO_GRAFANA_ADMIN_PASSWORD .env
curl: 'Cannot bind parameter Headers'	PowerShell's curl is Invoke-WebRequest: use the command in 2.4, or curl.exe.
Canary check says 'inconclusive'	Not enough evidence yet: it needs 200 canary requests from API clients and 5 minutes. Send traffic (2.4).
Grafana charts are flat	Wrong endpoint selected, or no traffic in the time range. Pick your endpoint; use Last 15 minutes.
Prediction returns 401 / 404 / 422 / 429	Bad key / wrong slug or project / inputs don't match the signature / rate limit.
Deployment stays in Provisioning	Docker is not running or a container failed: pnpm dev:ps, then pnpm test:smoke.
Page does not load at :3300	Window 2 (pnpm dev) is not running or still starting.

3.5 Stop and clean up

```
# in window 2: press Ctrl+C
pnpm dev:down       # stop the containers, keep all data
pnpm dev:clean      # stop AND delete all data (databases, storage), start fresh next time
```

3.6 About AWS

ModelOps Cloud has never used AWS: no account, no credentials, no deployment, no cost. The folder `infra\terraform\` describes how it *would* run on AWS (ECS Fargate, RDS, S3, least-privilege IAM). CI checks that code offline, and a written estimate (`infra\terraform\envs\demo\cost.md`) says what it would cost: about $0.29 per hour while running.

3.7 Where to read more

Document	Contents
README.md	Overview, quickstart, measured results
docs\demo-script.md	A 3-minute spoken demo
docs\10-local-development.md	Every command, port and service in detail
docs\02-, 05-, 06-*.md	Architecture, deployment lifecycle (states, retries, rollback), security and its tests
docs\milestones\	What each of the 12 milestones built and how it was tested